

22AIE201

Fundamentals of AI

A Project - Submitted by

Group 6

AIE - BATCH B

Name	Roll Number
AKHILESH MOHANASUNDARAM	(CB.SC.U4AIE23111)
NAREN B S	(CB.SC.U4AIE23152)
NITIN KRISHNA V	(CB.SC.U4AIE23156)
GAURAV MAHESH	(CB.SC.U4AIE23176)

PROJECT TOPIC:

Advanced Epilepsy detection through EEG signal analysis

As part of the 3rd Semester project



Department of Artificial Intelligence

AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF ENGINEERING

AMRITA VISHWA VIDHYAPEETHAM

COIMBATORE – 641 112

DECLARATION

We hereby declare that the project work entitled, “Advanced Epilepsy detection through EEG signal analysis”, submitted to Dr. Abhishek S, Assistant Professor, CEN, is our own work, based on our personal study and research. We have acknowledged all materials and sources used in its preparation, whether they be books, articles, reports, lecture notes, or any other kind of document, electronic or personal communication.

We also certify that this project has not previously been submitted for assessment in any academic capacity, and that we have not copied, in part or whole, or otherwise plagiarized the work of other persons. We confirm that we have identified and declared all possible conflicts that we may face.

Furthermore, we understand the importance of academic integrity and have adhered to the principles of honesty and fairness throughout this project. The methodologies and results presented are an honest representation of our efforts, and any assistance received has been appropriately acknowledged. We take full responsibility for the content of this work.

AIE Batch B

Group 6

Place: Coimbatore, Tamil Nadu, India

Date: 18 November 2024

ACKNOWLEDGEMENT

We would like to express our sincere appreciation to all those who contributed to the successful completion of this project. This endeavour would not have been possible without the collective efforts, dedication, and expertise of the following individuals.

We extend our gratitude to each member of the project team for their hard work, collaboration, and commitment to excellence. Their diverse skills and perspectives greatly enriched the project and ensured its successful execution.

We would like to acknowledge Dr. Abhishek S, Assistant Professor, CEN, for her collaboration and contributions to specific aspects of the project. Her expertise and cooperation significantly enhanced the overall quality of our work.

We are grateful for the support received from Amrita Vishwa Vidyapeetham. The resources, facilities, and conducive environment provided by the institution were instrumental in the successful execution of our project.

To our friends and family, who stood by us with patience and encouragement, we extend our heartfelt appreciation. Your understanding and support were invaluable during the demanding phases of this project.

In conclusion, each individual and entity mentioned above played a crucial role in the realization of this project. Our gratitude extends to everyone who contributed in various capacities, and we are proud to have worked with such a dedicated and talented team.

TABLE OF CONTENTS

Sl. No.	Content	Page Number
1.	Introduction	5 - 9
2.	Methodology	10 - 11

	– 2.1. Data Collection and Preprocessing – 2.2. Feature Extraction – 2.3. Data Labeling and Combination – 2.4. Train-Test Split and Scaling – 2.5. Class Balancing – 2.6. Machine Learning Model Training – 2.7. Deep Learning Model Implementation – 2.8. Model Evaluation and Comparison – 2.9. Validation and Performance Analysis – 2.10. Outcome	
3.	Implementation	12 - 16
4.	Result	17 - 24
5.	Inference	25
6.	Conclusion	26 - 27
7.	Reference - Bibiliography	28
8.	Source Code	29 - 45

1. Introduction :

Epilepsy, a chronic neurological disorder characterized by recurring seizures, affects millions of individuals worldwide, profoundly impacting their quality of life. Early and accurate detection of epileptic episodes is crucial for effective treatment, monitoring, and improving patient outcomes. Electroencephalography (EEG) signals, which capture the brain's electrical activity, serve as a critical tool for epilepsy diagnosis. However, the manual analysis of EEG data is time-consuming, prone to subjective errors, and often requires expert neurologists, making automated and efficient diagnostic methods an essential advancement. In this project, titled "**Advanced Epilepsy Detection through EEG Signal Analysis**," we aim to leverage the power of machine learning (ML) and deep learning (DL) to

automate the classification of EEG signals into focal and non-focal categories. Focal seizures originate from specific regions of the brain and often pose unique diagnostic challenges. The ability to differentiate between focal and non-focal seizures accurately can significantly enhance patient-specific treatment plans.

The project involves extracting a comprehensive set of features from EEG signals, including fractal dimensions (Higuchi and Katz), wavelet-based features, power spectral density, spectral entropy, and statistical measures such as mean, variance, skewness, and kurtosis. These features capture critical time-domain and frequency-domain characteristics of EEG signals, enabling machine learning models to classify them effectively.

To handle the class imbalance commonly found in EEG datasets, we utilize Synthetic Minority Oversampling Technique (SMOTE) for creating a balanced training dataset. Various machine learning classifiers, including Gradient Boosting, Random Forest, XGBoost, and Support Vector Machines (SVM), are trained and evaluated. Furthermore, we integrate a Long Short-Term Memory (LSTM)-based deep learning model to exploit temporal dependencies in the EEG data. The models are optimized using Randomized Search Cross-Validation, and their performances are compared based on accuracy and classification metrics.

This project not only explores state-of-the-art techniques in signal processing and machine learning but also addresses real-world challenges in medical diagnosis. By automating epilepsy detection, the system holds the potential to reduce diagnostic delays, assist neurologists in clinical decision-making, and ultimately improve the quality of life for individuals living with epilepsy.

2. Methodology :

The methodology for this project, "**Advanced Epilepsy Detection through EEG Signal Analysis,**" is designed to leverage advanced signal processing techniques, feature engineering, and machine learning algorithms for accurate classification of EEG data into focal and non-focal categories. The steps are outlined as follows:

2.1. Data Collection and Preprocessing

- **Data Sources:** EEG datasets comprising focal and non-focal seizure data are sourced from a reliable repository. Focal and non-focal signals are stored in separate folders for efficient loading and labeling.
- **Preprocessing:**
 - Raw EEG signals are read from .txt files and flattened for analysis.
 - Signals are normalized to remove inconsistencies arising from amplitude variations.
 - Noise and artifacts are minimized using appropriate filters (e.g., low-pass or bandpass filtering).

2.2. Feature Extraction

To extract meaningful information from the EEG signals, the following features are computed:

- **Fractal Dimensions:**
 - Higuchi Fractal Dimension (HFD) and Katz Fractal Dimension (KFD) to capture the signal's complexity.
- **Wavelet Features:**
 - Discrete Wavelet Transform (DWT) is applied to decompose signals into frequency bands, and statistical features like mean and standard deviation are derived from the coefficients.
- **Statistical Measures:**
 - Metrics like mean, variance, skewness, kurtosis, and root mean square (RMS) are calculated to characterize the signals.
- **Power Spectral Analysis:**
 - Power Spectral Density (PSD) and Bandpower across frequency bands (Delta, Theta, Alpha, Beta, Gamma) are computed.
- **Entropy Measures:**
 - Spectral Entropy and Hurst Exponent to quantify randomness and persistence in the signals.
- **Dynamic Features:**
 - Singular Value Decomposition (SVD) and Dynamic Mode Decomposition (DMD) features are derived to capture temporal patterns.

- **Other Features:**
 - Sevcik Fractal Dimension and Zero-Crossing Rate (ZCR) are included for additional signal characterization.

2.3. Data Labeling and Combination

- Labels are assigned to the focal and non-focal datasets (1 for focal and 0 for non-focal).
- Feature matrices and corresponding labels are combined into a single dataset for model training and testing.

2.4. Train-Test Split and Scaling

- The dataset is split into training and testing sets (94%-6% split).
- Features are standardized using `StandardScaler` to ensure uniformity in scale for all features.

2.5. Class Balancing

- To address the imbalance between focal and non-focal samples, the Synthetic Minority Oversampling Technique (SMOTE) is applied to the training set. This creates synthetic samples for the minority class, ensuring balanced training data.

2.6. Machine Learning Model Training

The following models are trained and optimized using Randomized Search Cross-Validation (CV):

- **Gradient Boosting Classifier**
- **Random Forest Classifier**
- **XGBoost Classifier**
- **Support Vector Machine (SVM)**

Each model is tuned for hyperparameters, and the best configuration is selected based on cross-validation accuracy.

2.7. Deep Learning Model Implementation

- A Long Short-Term Memory (LSTM) neural network is implemented to exploit temporal dependencies in the EEG data.
- The architecture includes:
 - Input layer followed by LSTM layers to process sequential data.
 - Dense and dropout layers for fully connected operations and regularization.
 - Output layer with a sigmoid activation function for binary classification.

2.8. Model Evaluation and Comparison

- Each model is evaluated on the test set using the following metrics:
 - **Classification Report:** Precision, Recall, F1-Score.
 - **Accuracy Score:** Overall classification performance.
- Comparisons are made between machine learning and deep learning models to identify the most effective approach for epilepsy detection.

2.9. Validation and Performance Analysis

- Performance is validated using unseen data to ensure generalizability.
- Results are analyzed to identify strengths and weaknesses of each model, particularly focusing on sensitivity for detecting focal seizures.

2.10. Outcome

The final system integrates the best-performing model to classify EEG data into focal and non-focal categories, offering a robust, automated solution for epilepsy detection. This methodology ensures reliability, scalability, and clinical applicability of the developed system.

3. Implementation :

The implementation of this project, "**Advanced Epilepsy Detection through EEG Signal Analysis**," involves several well-defined steps that span data preprocessing, feature extraction, model training, and evaluation. Below is the detailed implementation process, structured for a technical report.

3.1. Data Preprocessing

- **Data Loading:**
 - EEG data files are loaded from two separate directories: one for focal seizures and another for non-focal seizures.
 - Each file is read, and the signal is flattened into a 1D array for processing.
- **Normalization:**
 - The signals are normalized to ensure uniform scaling across all samples, minimizing variations due to differing recording conditions.
- **Noise Reduction:**

- Basic filtering techniques, such as bandpass filters, are applied to remove artifacts and focus on the frequency bands relevant to epilepsy.

3.2. Feature Extraction

A comprehensive set of features is extracted from the preprocessed EEG signals to characterize their temporal, spectral, and statistical properties. These include:

- **Fractal Dimensions:**
 - **Higuchi Fractal Dimension (HFD)** and **Katz Fractal Dimension (KFD)** are computed to quantify the complexity of the signals.
- **Wavelet Decomposition:**
 - Using the Discrete Wavelet Transform (DWT) with the Daubechies (db4) wavelet, the signal is decomposed into frequency sub-bands. Statistical features like mean and standard deviation are extracted from the wavelet coefficients.
- **Power Spectral Analysis:**
 - Power Spectral Density (PSD) is calculated using Welch's method to measure the energy distribution across frequencies.
 - Bandpower is computed for specific EEG frequency bands: Delta, Theta, Alpha, Beta, and Gamma.
- **Entropy Measures:**
 - Spectral Entropy is calculated to measure the randomness of the signal.
 - Hurst Exponent is derived to analyze the persistence or anti-persistence of the time series.
- **Statistical Features:**
 - Metrics such as mean, variance, skewness, kurtosis, root mean square (RMS), and zero-crossing rate (ZCR) are computed to capture signal characteristics.
- **Dynamic Features:**
 - Singular Value Decomposition (SVD) and Dynamic Mode Decomposition (DMD) are applied to extract features related to temporal patterns.

These features are combined into a feature matrix, with rows representing individual samples and columns representing extracted features.

3.3. Data Labeling and Splitting

- Labels are assigned to the data: 1 for focal seizures and 0 for non-focal seizures.
- The feature matrix and labels are combined into a single dataset and split into training and testing sets using an **80-20 split**.
- The training set is further balanced using **SMOTE** to generate synthetic samples for the minority class.

3.4. Machine Learning Model Training

- The following machine learning classifiers are implemented:
 - **Gradient Boosting Classifier**
 - **Random Forest Classifier**
 - **XGBoost Classifier**
 - **Support Vector Machine (SVM)**
- Each model is optimized using **RandomizedSearchCV**, which tunes hyperparameters by evaluating a predefined range of values using cross-validation.
- The best-performing hyperparameters for each model are recorded, and the models are trained on the balanced training dataset.

3.5. Deep Learning Model Implementation

- A deep learning model based on **Long Short-Term Memory (LSTM)** is implemented to capture temporal patterns in EEG data:
 - **Architecture:**
 - Input Layer: Accepts sequential data.
 - LSTM Layers: Extract temporal features.
 - Dense Layers: Perform fully connected operations.
 - Dropout Layers: Regularize the model and prevent overfitting.
 - Output Layer: A single neuron with a sigmoid activation function for binary classification.
 - The model is compiled using the Adam optimizer and binary cross-entropy loss function.
 - Training is performed over multiple epochs with early stopping to prevent overfitting.

3.6. Model Evaluation

- Each model (machine learning and deep learning) is evaluated on the test set using:
 - **Classification Report:** Precision, recall, F1-score for each class.
 - **Accuracy Score:** Overall classification accuracy.
 - **Confusion Matrix:** Visual representation of true positives, true negatives, false positives, and false negatives.

3.7. Performance Comparison

- The performances of all models are compared based on key metrics:
 - Accuracy
 - Sensitivity (recall for focal seizures)
 - Precision
 - F1-Score
- The model with the best balance of accuracy and sensitivity is selected for deployment.

3.8. Deployment-Ready System

- The final model is saved using joblib (for machine learning models) or TensorFlow's model saving utility (for LSTM).
- The system is designed to accept new EEG signals, preprocess them, extract features, and classify them as focal or non-focal in real-time.

3.9. Tools and Libraries

- **Python:** Primary programming language.
- **Libraries:**
 - **Signal Processing:** NumPy, SciPy, PyWavelets
 - **Machine Learning:** Scikit-learn, XGBoost, imbalanced-learn
 - **Deep Learning:** TensorFlow/Keras

- **Data Handling:** Pandas, Matplotlib

4. Results :

5. Conclusion :

The project, "**Advanced Epilepsy Detection through EEG Signal Analysis**," demonstrates the potential of leveraging advanced signal processing techniques and machine learning models to automate the diagnosis of epilepsy. By extracting a diverse set of features from EEG signals—ranging from fractal dimensions and wavelet features to power spectral density and entropy measures—this study provides a comprehensive characterization of brain activity. These features enable the accurate differentiation between focal and non-focal seizures, a critical step in personalized epilepsy treatment.

The integration of machine learning classifiers such as Gradient Boosting, Random Forest, XGBoost, and Support Vector Machines, alongside a Long Short-Term Memory (LSTM) deep learning model, ensures robust classification of EEG signals. The use of SMOTE to address class imbalance further enhances the system's reliability, particularly in identifying focal seizures, which are often underrepresented in datasets. Comparative analysis of the models reveals that while traditional machine learning approaches offer efficiency and interpretability, the LSTM model excels in capturing temporal dependencies, leading to improved performance in certain scenarios.

This work not only showcases the effectiveness of combining feature engineering with advanced classification algorithms but also underscores the importance of a well-balanced dataset and rigorous evaluation metrics. The system has the potential to assist neurologists in making faster, more accurate diagnoses, ultimately improving patient care. Future directions include real-time implementation, integration with wearable EEG

devices, and extending the system's capability to predict seizures, thereby providing proactive solutions for epilepsy management. This project lays the foundation for a scalable, clinically relevant tool in modern healthcare technology.

6. References – Bibliography :

1. Abbasi, B. and Goldenholz, D.M., 2019. Machine learning applications in epilepsy. *Epilepsia*, Available at: <https://doi.org/10.1111/epi.16364>.
2. Mengni Zhou, Cheng Tian , Ting Hu Epileptic Seizure Detection Based on EEG Signals and CNN
<https://www.frontiersin.org/journals/neuroinformatics/articles/10.3389/fninf.2018.00095/full>
3. Truong, N. D., Nguyen, A. D., Kuhlmann, L., Bonyadi, M. R., Yang, J., & Ippolito, S. (2018). Convolutional neural networks for seizure prediction using intracranial and scalp EEG. *Epilepsy & Behavior*, 85, 112-118. <https://doi.org/10.1016/j.yebeh.2018.05.044>
4. Thodoroff, P., Pineau, J., & Lim, A. (2016). Learning robust features using deep learning for automatic seizure detection. *Conference on Machine Learning in Healthcare, Proceedings of Machine Learning Research*, 56, 178-190. <https://proceedings.mlr.press/v56/Thodoroff16.html>
5. Afshin Shoeibi , Marjane Khodatars, Navid Ghassemi. Epileptic Seizures Detection Using Deep Learning Techniques: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8199071/>

7. Source Code :

```
import numpy as np import os import warnings import pywt from
scipy import signal from scipy.stats import skew, kurtosis from
sklearn.model_selection import train_test_split,
RandomizedSearchCV from sklearn.preprocessing import
StandardScaler from sklearn.metrics import classification_report,
accuracy_score from sklearn.ensemble import
GradientBoostingClassifier, RandomForestClassifier from xgboost
import XGBClassifier from sklearn.svm import SVC from
imblearn.over_sampling import SMOTE from
tensorflow.keras.models import Sequential from
tensorflow.keras.layers import LSTM, Dense, Dropout

warnings.filterwarnings("ignore")

focal_folder_path = '/Users/gaurav/Downloads/Complete
set/Data_F_Ind_1_750' non_focal_folder_path =
'/Users/gaurav/Downloads/Complete set/Data_N_Ind_1_750'

def sevcik_fractal_dimension(data): L = np.sum(np.sqrt(1 +
np.diff(data)**2)) return L / len(data)
```

```
def wavelet_features(data, wavelet='db4', level=4): coeffs =
pywt.wavedec(data, wavelet, level=level) features = [np.mean(c) for
c in coeffs] + [np.std(c) for c in coeffs] return features
```

```
def hurst_exponent(data): N = len(data) T = np.arange(1, N + 1) Y =
np.cumsum(data - np.mean(data)) R = np.max(Y) - np.min(Y) S =
np.std(data) return R / S if S != 0 else 0
```

```
def higuchi_fractal_dimension(data, kmax=10): Lk = [] N = len(data)
for k in range(1, kmax + 1): Lmk = [] for m in range(k): Lmki = 0 for i in
range(1, int((N - m) / k)): Lmki += abs(data[m + i * k] - data[m + (i - 1) *
k]) norm_factor = (N - 1) / (int((N - m) / k) * k) Lmk.append(Lmki *
norm_factor) Lk.append(np.mean(Lmk)) hfd =
np.polyfit(np.log(range(1, kmax + 1)), np.log(Lk), 1)[0] return hfd
```

```
def katz_fractal_dimension(data): d =
np.max(np.sqrt((np.arange(len(data)) - 0)**2 + (data - data[0])**2)) L
= np.sum(np.sqrt(1 + np.diff(data)**2)) return np.log10(L) /
(np.log10(d) + np.log10(len(data)))
```

```
def power_spectral_density(data): freqs, psd = signal.welch(data,
fs=512) return np.mean(psd)
```

```
def spectral_entropy(data): power_spectrum =
np.abs(np.fft.fft(data))**2 power_spectrum /=
np.sum(power_spectrum) return -np.sum(power_spectrum *
np.log2(power_spectrum + 1e-10))
```

```
def bandpower(data, sf, band, window_sec=None): band =
np.asarray(band) low, high = band if window_sec: nperseg =
int(window_sec * sf) else: nperseg = None freqs, psd =
```

```

signal.welch(data, sf, nperseg=nperseg) freq_res = freqs[1] -
freqs[0] idx_band = np.logical_and(freqs >= low, freqs <= high) bp =
np.trapz(psd[idx_band], dx=freq_res) return bp

```

```

def svd_features(data): u, s, vh = np.linalg.svd(data.reshape(-1, 1),
full_matrices=False) return np.mean(s), np.std(s)

```

```

def dmd_features(data, rank=5): x = data[:-1].reshape(-1, 1) y =
data[1:].reshape(-1, 1) u, s, vh = np.linalg.svd(x,
full_matrices=False) s = np.diag(s) u_r = u[:, :rank] s_r =
s[:rank, :rank] vh_r = vh[:, :rank, :] A_tilde = u_r.T @ y @ vh_r.T @
np.linalg.inv(s_r) eigenvalues = np.linalg.eigvals(A_tilde) return
np.real(eigenvalues).mean(), np.imag(eigenvalues).mean()

```

```

def load_data_with_features(folder_path, limit=3750): file_list = [f
for f in os.listdir(folder_path) if f.endswith('.txt')][:limit] data_array =
[] for file_name in file_list: file_path = os.path.join(folder_path,
file_name) data = np.loadtxt(file_path, delimiter=',').flatten()

```

```

hfd = higuchi_fractal_dimension(data)
kfd = katz_fractal_dimension(data)
psd = power_spectral_density(data)
se = spectral_entropy(data)
sevcik = sevcik_fractal_dimension(data)
hurst = hurst_exponent(data)
wavelet_feat = wavelet_features(data)
mean_val, std_val = np.mean(data), np.std(data)
var_val, skew_val, kurt_val = np.var(data), skew(data),
kurtosis(data)
rms_val = np.sqrt(np.mean(data**2))

```

```

zcr_val = ((data[:-1] * data[1:]) < 0).sum() / len(data)
mean_svd, std_svd = svd_features(data)
real_dmd, imag_dmd = dmd_features(data)
sf = 250
delta_bp = bandpower(data, sf, [0.5, 4])
theta_bp = bandpower(data, sf, [4, 8])
alpha_bp = bandpower(data, sf, [8, 12])
beta_bp = bandpower(data, sf, [12, 30])
gamma_bp = bandpower(data, sf, [30, 100])

features = [
    hfd, kfd, psd, se, sevcik, hurst,
    delta_bp, theta_bp, alpha_bp, beta_bp, gamma_bp,
    mean_val, std_val, var_val, skew_val, kurt_val,
    rms_val, zcr_val, mean_svd, std_svd, real_dmd, imag_dmd
] + wavelet_feat
data_array.append(features)
return np.array(data_array)

from sklearn.utils import shuffle

focal_data = load_data_with_features(focal_folder_path)
non_focal_data = load_data_with_features(non_focal_folder_path)

focal_labels = np.ones(len(focal_data)) non_focal_labels =
np.zeros(len(non_focal_data)) X = np.vstack((focal_data,
non_focal_data)) y = np.concatenate((focal_labels,
non_focal_labels))

```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1,
random_state=42) scaler = StandardScaler() X_train_scaled =
scaler.fit_transform(X_train) X_test_scaled =
scaler.transform(X_test)
```

```
smote = SMOTE(random_state=42) X_train_balanced,
y_train_balanced = smote.fit_resample(X_train_scaled, y_train)
```

```
X_train_balanced, y_train_balanced = shuffle(X_train_balanced,
y_train_balanced, random_state=42)
```

```
import tensorflow as tf from tensorflow.keras.models import
Sequential from tensorflow.keras.layers import LSTM, Dense,
Dropout, BatchNormalization from tensorflow.keras.optimizers
import Adam from tensorflow.keras.regularizers import l2 from
sklearn.metrics import accuracy_score, classification_report
```

```
def lstm_model(input_shape): model = Sequential()
```

```
model.add(LSTM(256, return_sequences=True,
input_shape=input_shape, kernel_regularizer=l2(0.001)))
model.add(BatchNormalization())
model.add(Dropout(0.4))
```

```
model.add(LSTM(128, return_sequences=True,
kernel_regularizer=l2(0.001)))
model.add(BatchNormalization())
model.add(Dropout(0.3))
```

```
model.add(LSTM(64, return_sequences=False,
kernel_regularizer=l2(0.001)))
```

```

model.add(BatchNormalization())
model.add(Dropout(0.3))

model.add(Dense(64, activation='relu',
kernel_regularizer=l2(0.001)))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))

optimizer = Adam(learning_rate=0.0005, clipvalue=1.0)
model.compile(optimizer=optimizer, loss='binary_crossentropy',
metrics=['accuracy'])
return model

```

```

X_train_lstm =
X_train_balanced.reshape((X_train_balanced.shape[0], 1,
X_train_balanced.shape[1])) X_test_lstm =
X_test_scaled.reshape((X_test_scaled.shape[0], 1,
X_test_scaled.shape[1]))

final_lstm_model = build_final_lstm_model((X_train_lstm.shape[1],
X_train_lstm.shape[2]))

history = final_lstm_model.fit( X_train_lstm, y_train_balanced,
epochs=1000, batch_size=128, verbose=2 )

y_pred = (final_lstm_model.predict(X_test_lstm) > 0.5).astype(int)
print("Final LSTM Model Accuracy:", accuracy_score(y_test,
y_pred)) print(classification_report(y_test, y_pred))

```

```
gb_params = { 'n_estimators': [600], 'learning_rate': [0.2, 0.25],  
'max_depth': [13], 'min_samples_split': [5], 'min_samples_leaf':  
[10], 'subsample': [1.0], 'max_features': ['log2'] }  
evaluate_model_random_search(GradientBoostingClassifier(),  
gb_params, 'Gradient Boosting')
```

```
xgboost_params = { 'n_estimators': [800], 'learning_rate': [0.15],  
'max_depth': [13], 'min_child_weight': [4], 'subsample': [1],  
'colsample_bytree': [1.0] }  
evaluate_model_random_search(XGBClassifier(eval_metric='logloss'),  
xgboost_params, 'XGBoost')
```

```
rf_params = { 'n_estimators': [300], 'max_depth': [20],  
'min_samples_split': [2], 'min_samples_leaf': [2], 'max_features':  
['log2'], 'bootstrap': [False] }  
evaluate_model_random_search(RandomForestClassifier(),  
rf_params, 'Random Forest')
```

```
svm_params = [{'kernel': ['rbf'], 'C': [10, 50], 'gamma': ['scale', 0.1,  
1]}] evaluate_model_random_search(SVC(), svm_params, 'SVM')
```